

1. Black-Board architecture

What is Black-Board architecture?

Blackboard Architecture란 독립적인 **Knowledge Source(모듈)**들이 문제 해결을 위해 협력하는 구조이다. 블랙보드라는 저장소에 문제를 해결할 수 있는 모듈이 해답의 일부를 기록한다. 이 과정을 계속 반복하여 수행하면, **최종 해결책**을 도출할 수 있다.

블랙보드 아키텍처의 특징은 **모듈들의 활성화 순서가 블랙보드의 상태에 따라 결정**된다는 것이다. 또한 각 모듈들은 서로 직접적으로 통신 하지 않으며, 블랙보드를 통해서만 통신이 가능하다.

이름은 사람들이 칠판(black-board)을 중심으로 협력하는 방식에서 유래되었다.

What vocabulary does the Black-Board architecture have for its components and connectors?

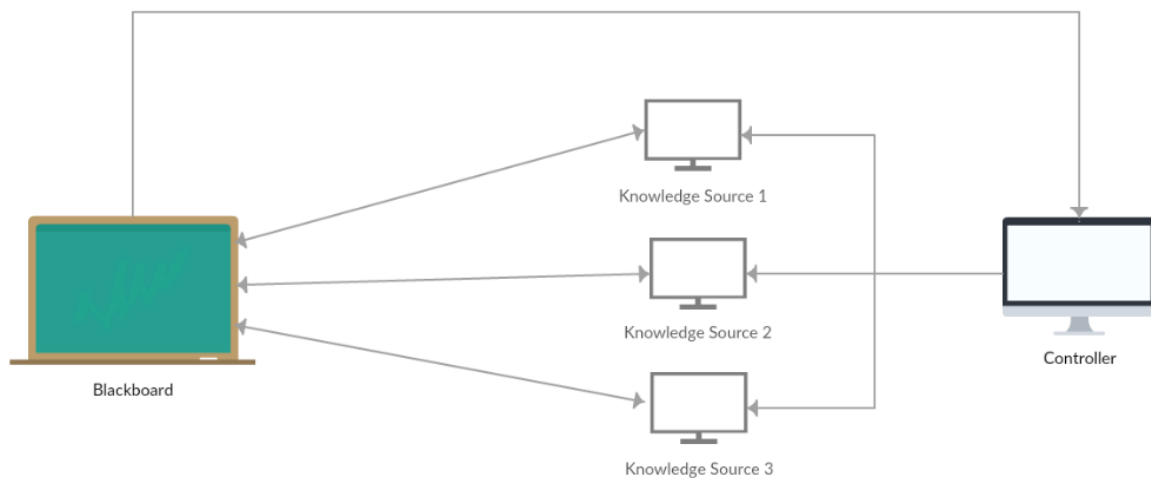
Component:

- Black-Board (중앙 데이터 저장소),
 - 입력, 중간결과 등을 저장하는 전역에서 접근 가능한 데이터베이스.
 - 현재 시스템의 상태를 나타낸다.
- Knowledge source(모듈),
 - 특정 문제를 해결하는 지식을 갖춘 독립적인 모듈
 - 어떤 조건이 충족되면 활성화되어 블랙보드에 문제 해결책을 추가해 해결을 진행해 나간다.
- Controller(컨트롤러)
 - 여러 모듈이 동시에 블랙보드에 접근하려할 때, 가장 적합한 모듈을 선택해 실행한다.
 - 블랙보드의 상태가 바뀔 때 마다 가장 적합한 모듈을 선택하여 문제 해결진행.

Connector:

- 구현 방식에 따라 다양하다. 블랙보드가 단순 데이터베이스라면, 쿼리 자체가 커넥터가 될 수 있고, 모든 모듈이 동일한 기기에 있다면 메모리 참조가 커넥터 역할을 할 수 있다. 복잡한 경우에는 API 호출이 커넥터 역할을 수행할 수 있다.

What topological constraints does it impose (diagram)?



구성 요소: 블랙보드(Blackboard), 여러 지식 소스(Knowledge Sources), 하나의 컨트롤러(Controller).

1. Knowledge source: 서로 직접 통신이 불가하며, 블랙보드를 통해서만 통신한다. 블랙보드를 수정할 수 있는 유일한 컴포넌트이다.
2. Controller: 블랙보드를 지속적으로 모니터링한다. 현재 블랙보드 상태에 따라 블랙보드에 접근할 다음 모듈을 선택해 실행한다.
3. Black-Board: 컨트롤러와 지식 소스 모두 블랙보드에 접근 가능하지만, 블랙보드 수정은 지식 소스만 할 수 있다.

What are the advantages and disadvantages?

장점

- 데이터 공유: 블랙보드에 모든 데이터가 저장되어 모듈 간 데이터 공유가능.
- 유연한 문제 해결: 문제 해결법이 정해져 있지 않고 각 모듈이 상황에 맞게 해결.
- 확장성: 각 모듈들이 독립적이므로 쉽게 추가/제거 가능. 다른 블랙보드에서 사용가능.
- 효율성: 모듈이 독립적으로 동작할 수 있어 병렬 처리가 가능..
- 큰 문제 분할: 각 모듈이 블랙보드에 부분적으로 해결책을 추가하여 다른 모듈이 활용하여 전체 문제를 해결 가능.

단점

- 복잡성: 구현이 매우 복잡하며, 각 모듈 관리가 어렵다.
- 테스트: 각 상황에 따라 다른 방식으로 문제를 해결하므로, 매번 다른 결과가 나올 수 있다.
- 문제해결: 전체 문제 해결이 되지 않고, 일부분만 해결될 수 있다.
- 데이터: 데이터가 자주 변경되는 경우 효율적이지 않다.

What are the examples in practice?

- 문제를 단순한 하위 문제들로 나누어 해결해야 하는 사례.
- 비결정적 문제: 상황에 따라 해결책이 달라지는 사례.
- 다양한 해결 방법이 있는 사례.
- 사물 인터넷 (IoT): 각 센서가 개별 knowledge(모듈)로 작동하며, 하나의 저장소를 통해 협력해야하는 경우.
- 경험을 통해 서서히 문제를 해결하는 경우.

지원하는 비기능 요구사항 (Non-Functional Properties)

- 재사용성: 다른 블랙보드에 모듈이 사용될 수 있음.
- 확장성: 모듈이 독립적이기 때문에 확장성이 뛰어나.
- 모듈이 독립적이기 때문에, 모듈이 다운되어도 교체가 용이하고, 점차적으로 모듈을 업그레이드 가능.

지원되지 않는 비기능 요구사항 (Non-Functional Properties)

- 신뢰성: 블랙보드가 다운되면 전체 시스템에 문제.
- 이식성: 복잡한 시스템으로 인해 환경이 다른 경우 시스템 이식이 불가능.
- 성능: 상황에 따라 결과가 달라지므로, 매번 성능이 달라질 수 있다.

2. Client-Server architecture

What is the Client-Server architecture?

클라이언트-서버 아키텍처는 서버가 리소스와 서비스를 호스팅하고 관리하며, 클라이언트가 이를 소비하는 컴퓨팅 모델이다.

클라이언트는 네트워크를 통해 중앙 서버와 연결된다. 모든 요청과 서비스가 네트워크를 통해 전달된다. 클라이언트가 서버에 데이터를 요청하고, 서버가 요청을 처리한 후 결과를 클라이언트로 반환하는 흐름이 반복된다.

What vocabulary does the Client-Server architecture have for its components and connectors?

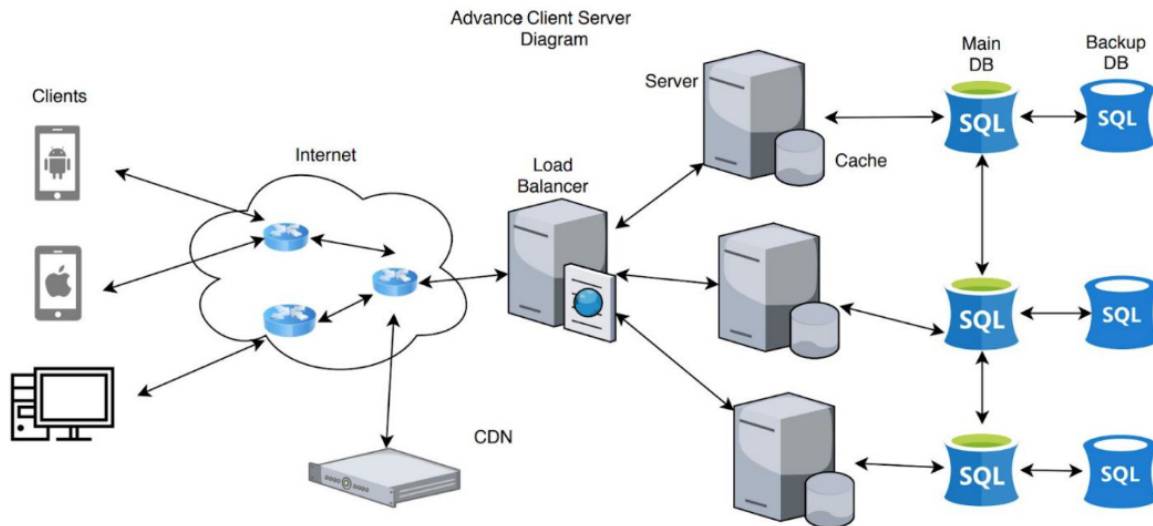
Component:

- 클라이언트(Client):
 - 입력을 받아 서버에 요청을 보내는 소프트웨어나 애플리케이션.
- 서버(Server):
 - 클라이언트의 요청을 받아 요청사항을 처리하는 소프트웨어.

Connector:

- 로드 밸런서(Load Balancer):
 - 클라이언트와 서버 사이에서 서버로 들어오는 요청을 여러 서버로 분산하여 자원 사용을 최적화하는 역할을 한다. (한 서버에 몰리지 않도록)
- 네트워크 계층 프로토콜(Network Layer Protocols):
 - TCP/IP와 같은 네트워크 프로토콜로, 클라이언트와 서버의 통신이 가능하게 한다.

What topological constraints does it impose (diagram)?



- 1. 단방향 데이터 흐름:** 데이터 흐름이 단방향이며, 클라이언트가 데이터를 요청하고 서버가 응답하는 구조를 따른다. 즉, 데이터의 흐름 자체는 단방향으로 사이클을 이룬다.
- 2. 클라이언트 간 직접 통신 금지:** 클라이언트는 서로 직접 통신할 수 없고, 반드시 서버를 통해 데이터를 요청하거나 전달받아야 한다. 즉, 클라이언트 간의 상호작용을 제한한다.
- 3. 프로토콜을 통한 통신:** 서버와 클라이언트가 프로토콜을 통해 통신하므로, 데이터 교환 방식에 구체적인 방법을 제시한다.

What are the advantages and disadvantages?

단점

1. 서버 의존성: 서버가 많은 클라이언트 요청을 동시에 처리해야 할 경우 서버가 과부하

될 수 있다. 특히, 서버의 자원이 충분하지 않으면 응답 지연이나 서비스 중단이 발생할 수 있다. 이런 경우 모든 클라이언트는 서버가 다운되거나 장애가 발생하면 서버에 접근할 수 없게 된다.

2. 네트워크 의존성: 네트워크를 통해 클라이언트와 서버가 연결되므로, 네트워크가 불안정하거나 속도가 느릴 경우 성능에 영향을 받을 수 있다.

3. 복잡한 보안 관리: 중앙 서버에 중요한 데이터가 모이기 때문에 보안 위협이 집중될 수 있다. 서버가 공격을 받을 경우, 전체 데이터가 위협에 처할 수 있다.

What are the examples in practice?

- 클라이언트와 서버의 기능을 분리하는 방식의 요구사항에 적합하다. 로드 밸런싱 등의 기법을 통해 대규모 시스템에서 좋은 성능을 제공한다. 여러 사용자 요청을 동시에 처리하는 Web, 모바일 Sns 서비스, 온라인 게임 등에 적합하다.

지원하는 비기능 요구사항 (Non-Functional Properties)

- 확장성: 서버를 더 추가하거나 서버 자체를 키우는 방식의 확장 가능.
- 가용성: 오랜 시간 동안 작동할 수 있으며, 서버를 여러 개 두어 서로 교체하며 작동할 수 있다.

지원되지 않는 비기능 요구사항 (Non-Functional Properties)

- 신뢰성: 많은 클라이언트가 동시에 요청을 보내면 서버가 과부하 될 수 있고, 이로 인해 서버가 다운되면 시스템이 중단되는 위험이 있다.

3. Peer-to-Peer architecture

What is the Peer-to-Peer architecture?

서버-클라이언트 아키텍처에서는 클라이언트와 서버의 역할이 명확히 구분되지만, Peer-to-Peer, P2P 아키텍처에서는 모든 노드(피어)가 동시에 클라이언트와 서버의 역할을 수행한다. 모든 피어는 하나의 중앙 노드에 연결되지 않으며, 서로 직접 통신하며 정보를 주고받는다.

What vocabulary does the Peer-to-Peer architecture have for its components and connectors?

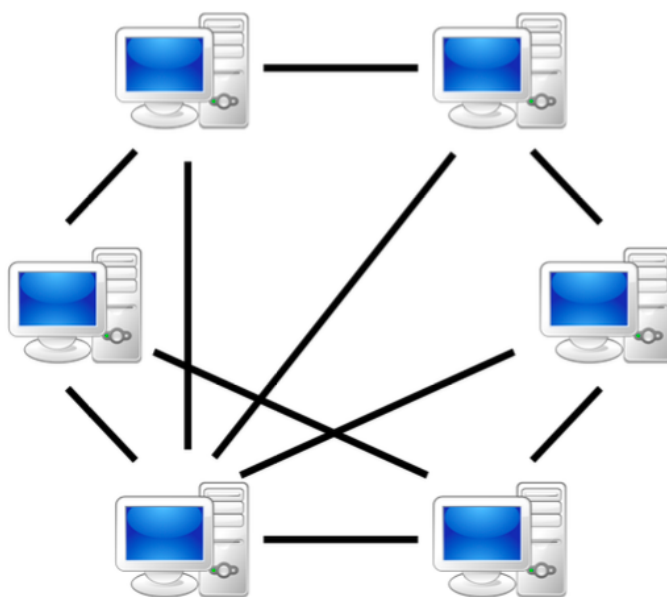
Component:

- 피어(Peer) (또는 피어 노드):
 - 클라이언트와 서버 역할을 동시에 수행한다.

Connector:

- P2P 프로토콜:
 - 피어 간의 통신을 위한 프로토콜이다. 이 프로토콜을 통해 공유한 데이터에만 접근이 가능하다.
 - 예시로 BitTorrent, Gnutella, Tor, I2P, Bitcoin, Ethereum 등이 존재한다.

What topological constraints does it impose (diagram)?



1. 피어 간 데이터 접근 제한:

피어는 서로 공유한 데이터에만 접근할 수 있다. 특정 데이터에 접근하려면, 그 데이터를 갖고 있는 다른 피어가 네트워크(p2p 프로토콜로 형성된 네트워크)에 존재해야 한다. 또한, 두 피어 간에 연결을 형성하기 위해서는 접근 가능한 노드 경로(p2p 프로토콜로 형성된 네트워크 경로)가 존재해야 합니다.

2. 새로운 피어의 연결:

피어가 처음 네트워크에 연결될 때, 다른 노드들의 존재 정보를 얻기 위해 부트 스트랩 노드의 주소를 알아야 한다. 부트스트랩 노드란 일부 노드의 주소를 알고있어, 새로운 노드가 연결점이 필요할 경우 다른 노드의 주소를 제공한다. 그렇기에 새로운 노드가 네트워크 연결을 형성하려면 시간이 필요할 수 있다.

3. P2P 네트워크의 연결 구조:

P2P 네트워크는 반드시 모든 노드가 완전히 연결된 형태를 가질 필요는 없다. 일부 노드는 네트워크의 대부분 노드와 연결될 수 있지만, 다른 노드는 소수의 노드들 과만 연결될 수 있다. 만약 두 노드가 직접 연결되어 있지 않더라도, 중간에 다른 노드를 통한 경로가 존재한다면, 정보를 교환할 수 있다.

What are the advantages and disadvantages?

장점

확장성: 피어가 자유롭게 참여하거나 떠날 수 있다. 이에 따라 네트워크가 유연하게 확장되거나 축소될 수 있다.

성능: 피어가 추가되면 네트워크 자원이 증가하여 여러 피어들이 같은 데이터를 공유하는 경우 더 빠르게 데이터 전송이 가능하다.

비용: 인프라나 서버가 필요하지 않기 때문에, 운영비용이 적다.

단점

보안성: 피어가 잘못된 데이터를 공유하여 다른 피어에 위협을 가할 수 있어 네트워크가 보안에 취약하다.

비인기 데이터: 요청한 데이터를 가진 피어가 네트워크에 존재해야만 해당 데이터를 받을 수 있다. 특정 데이터를 가진 노드가 없다면 접근이 불가능하다.

중앙 관리 부재: 프로토콜을 변경하거나 업데이트할 때 네트워크 전체가 일관되게 대응하기 어렵다. 또한 불법 활동을 제어하는 데 어렵다. 일부 피어가 새로운 프로토콜을 사용하면 다른 피어들과 통신이 불가능 하며, 네트워크가 분열될 위험이 있다.

What are the examples in practice?

- 분산컴퓨팅 시스템
- 블록체인 시스템
- 검열을 피해야 하는 통신 등

참고자료: 수업자료, ChatGPT 4.o, Document of university of Waterloo,

<https://en.wikipedia.org/wiki/Peer-to-peer#Architecture>

[https://en.wikipedia.org/wiki/Blackboard_\(design_pattern\)](https://en.wikipedia.org/wiki/Blackboard_(design_pattern))

https://en.wikipedia.org/wiki/Client%E2%80%93server_model